JOBSHEET 05

TEKNIK STYLING PADA NEXT.JS

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_05

D. Langkah Praktikum

1. Global CSS

a. File Global

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html,
body {
  max-width: 100vw;
  overflow: hidden;
}


a {
  color: inherit;
  text-decoration: none;
}

.navbar {
  width: 100%;
  height: 60px;
  background-color: #333;
  color: white;
  display: flex;
  align-items: center;
  padding: 0 20px;
}
```

b. Import Global CSS

// pages/_app.tsx

import "@/styles/globals.css";

```typescript
import '@/styles/globals.css'
import type { AppProps } from 'next/app'
import AppShell from '@/components/Appshell'
import Navbar from '@/components/index'

export default function App({ Component, pageProps }: AppProps) {
  return (
    <AppShell>
      <Component {...pageProps} />
    </AppShell>
  )
}
```

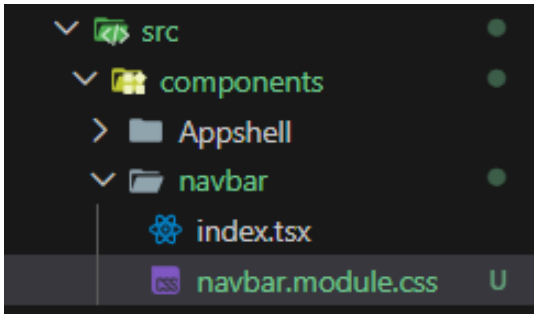
Catatan:

- Global CSS hanya bisa di-import di _app.tsx
- Berlaku untuk seluruh aplikasi

2. CSS Module (Local Scope)

a. Struktur Komponen Navbar

```
src/components/layout/Navbar/
├── index.tsx
└── Navbar.module.css
```



b. File CSS Module

- Modifikasi global.css

```
globals.css M ×   _app.tsx   navbar.module.css U
my-app-05 > src > styles > globals.css > a
1
2
3   * {
4     box-sizing: border-box;
5     margin: 0;
6     padding: 0;
7   }
8
9   html,
10  body {
11    max-width: 100vw;
12    overflow: hidden;
13  }
14
15
16  a {
17    color: inherit;
18    text-decoration: none;
19  }
20
21
22
```

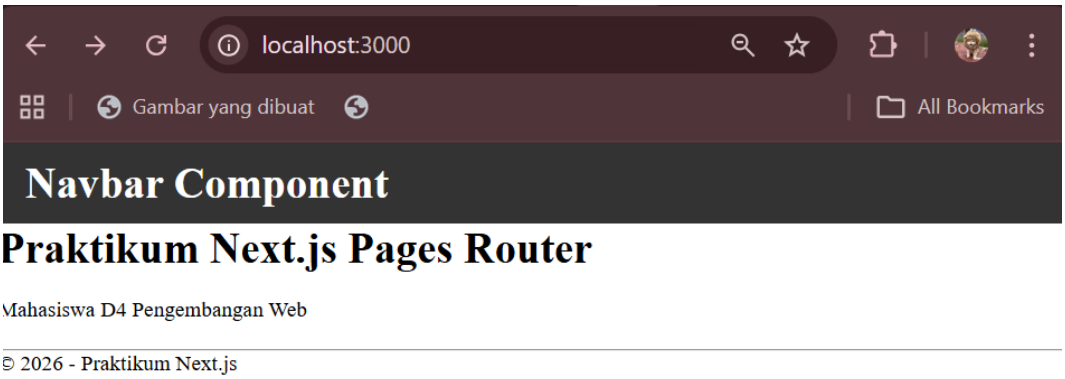
- Modifikasi navbar.module.css

```
my-app-05 > src > components > navbar > navbar.module.css > .navbar
1   .navbar {
2     width: 100%;
3     height: 60px;
4     background-color: #333;
5     color: white;
6     display: flex;
7     align-items: center;
8     padding: 0 20px;
9   }
```

c. Pemanggilan di Komponen
   - Modifikasi kode pada index.tsx pada folder navbar

```
globals.css M    index.tsx M ×    navbar.module.css U
my-app-05 > src > components > navbar > index.tsx > Navbar
1  import styles from './navbar.module.css';
2
   Windsurf: Refactor | Explain | Generate JSDoc | ×
3  const Navbar = () => {
4    return (
5      <div className={styles.navbar}>
6          <div>
7              <h1>Navbar Component</h1>
8          </div>
9      </div>
10   );
11 }
12
13 export default Navbar;
```

- Jalankan browser



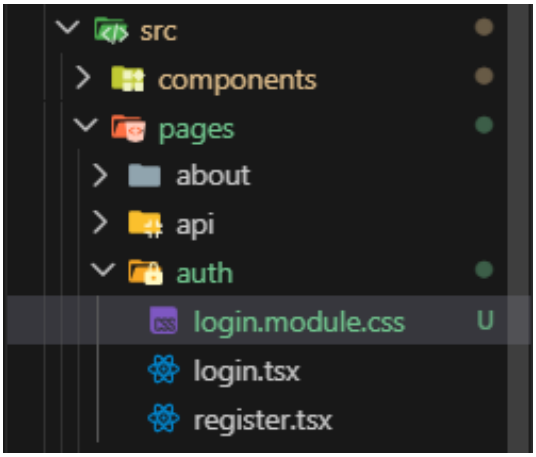
Keunggulan CSS Module:

• Scope lokal

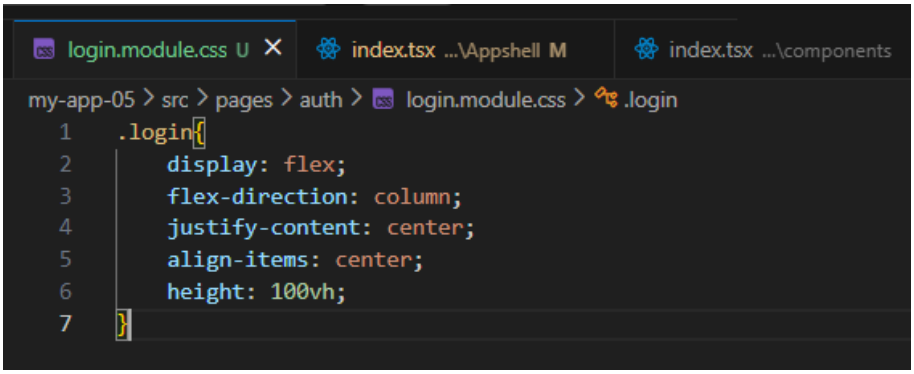
• Tidak bentrok antar class

• Cocok untuk komponen reusable

3. Styling untuk Pages (CSS Module)
   a. Contoh Login Page
   - Tambahkan login.module.css pada folder auth



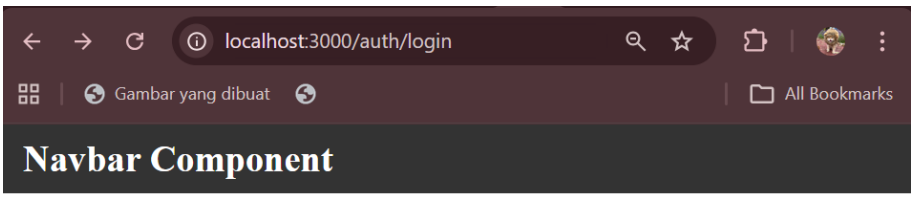
- Modifikasi login.module.css

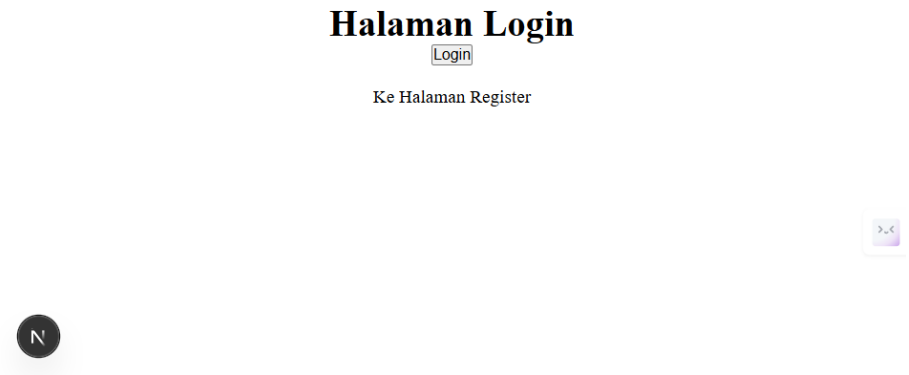


- Modifikasi login.tsx
  - Tambahkan import styles
  - Tambahan classname = { styles.login}

```
import Link from "next/link";
import { useRouter } from "next/router";
import styles from "./login.module.css";

const halamanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // Simpan status login
        localStorage.setItem("isLogin", "true");

        // logic login disini
        push("/produk");
    }
    return (
        <div className={styles.login}>

            <h1>Halaman Login</h1>

            {/* <button onClick={handleLogin}>Login</button> <br /> */}
            {/* <button onClick={() => push('/produk')}>Login</button> <br /> */}

            {/* Navigasi Imperatif */}
            <button onClick={() => handleLogin()}>Login</button> <br />

            {/* Navigasi Link */}
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default halamanLogin;
```

- Jalankan browser

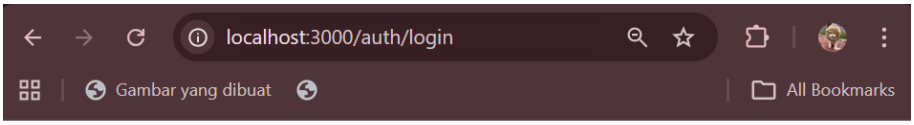




4. Conditional Rendering Navbar (Tanpa Navbar di Login)
    - Modifikasi index.tsx pada folder Appsheel
        - import { useRouter } from "next/router";
        - const disableNavbar = ["/auth/login", "/auth/register"];
        - const {pathname} = useRouter();
        - {!disableNavbar.includes(router.pathname) && <Navbar />}

```
import Navbar from "@/components/navbar";
import { useRouter } from "next/router";

const disableNavbar = ["/auth/login", "/auth/register"];
type AppShellProps = {
    children: React.ReactNode;
}

const AppShell = (props:AppShellProps) => {
    const { children } = props;
    const { pathname } = useRouter();
    return (
        <main>
            {!disableNavbar.includes(pathname) && <Navbar />}
        {children}
        <footer style={{ marginTop: "20px" }}>
            <hr />
            <p> © 2026 - Praktikum Next.js</p>
        </footer>
        <div>
        </div>
        </main>
    );
    };

export default AppShell;
```

- Jalankan browser



Halaman Login

Login

Ke Halaman Register

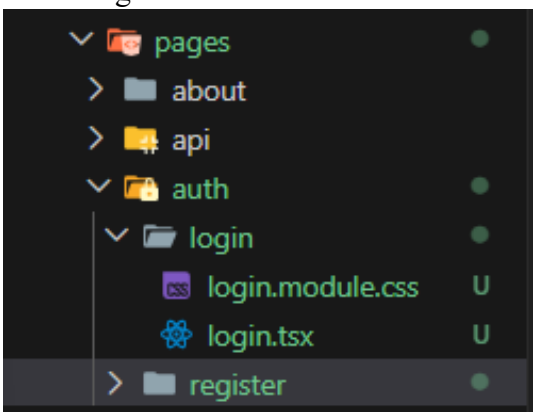
Insight:

- Gunakan router.pathname jika ada query parameter
  - const {pathname} = useRouter();
  - Darimana kah pathname => lakukan console.log pada router

```
const { children } = props;
const { pathname } = useRouter();
const router = useRouter();
console.log(router);  ×2 ᵇ { pathname: '/auth/login', route: '/auth/login', query: {}, asPath: '/auth/login', components: { '/auth/login': {...}, '/_app'...
```

- Jangan gunakan asPath jika ada query parameter

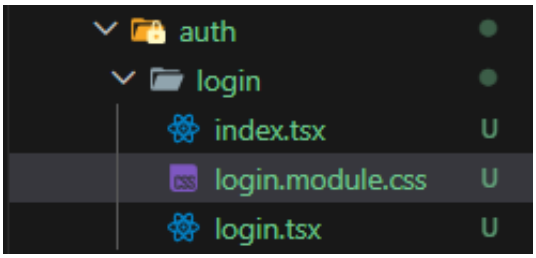
5. Refactoring Struktur Project (Best Practice)
   a. Struktur Awal (Kurang Rapi)
   pages/login.tsx
   styles/login.module.css
   b. Struktur Refactor (Disarankan)
   pages/auth/login.tsx
   src/views/auth/Login/
   ├── index.tsx
   └── Login.module.css



- Modifikasi login.module.css pada folder view/auth/login/

```
my-app-05 > src > pages > auth > login > login.module.css > .login
1  .login{
2      display: flex;
3      flex-direction: column;
4      justify-content: center;
5      align-items: center;
6      height: 100vh;
7  }
```

- Login.module.css pada folder pages/auth dihapus



- Modifikasi login.tsx pada folder pages/auth Modifikasi login.tsx pada folder pages/auth

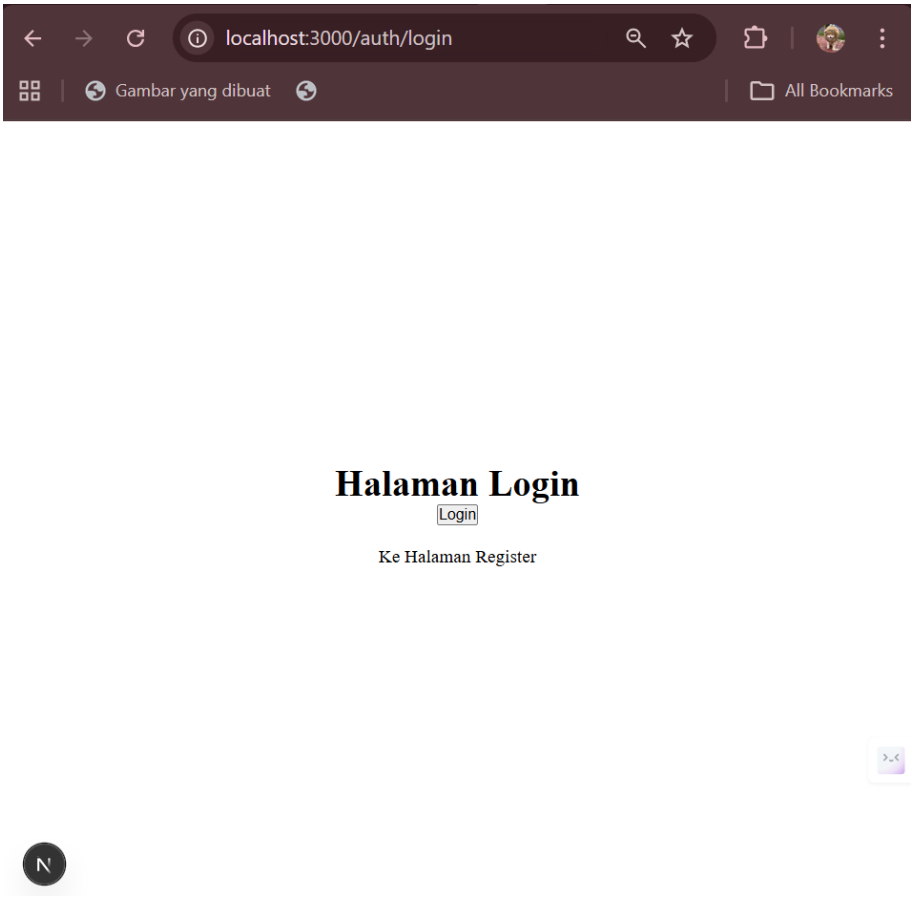
my-app-05 > src > pages > auth > login > login.tsx > default

```
import TampilanLogin from "@/pages/auth/login/index";

Windsurf: Refactor | Explain | Generate JSDoc | X
const halamanLogin = () => {

return (
<>
<TampilanLogin />
</>
);
};

export default halamanLogin;
```

- Modifikasi index.tsx pada folder views/auth/login

```
import Link from "next/link";
import { useRouter } from "next/router";
import styles from "./login.module.css";

const TampilanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // Simpan status login
        localStorage.setItem("isLogin", "true");

        // logic login disini
        push("/produk");
    }
    return (
        <div className={styles.login}>

            <h1>Halaman Login</h1>

            {/* <button onClick={handleLogin}>Login</button> <br /> */}
            {/* <button onClick={() => push('/produk')}>Login</button> <br /> */}

            {/* Navigasi Imperatif */}
            <button onClick={() => handleLogin()}>Login</button> <br />

            {/* Navigasi Link */}
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default TampilanLogin;
```

- Jalankan browser



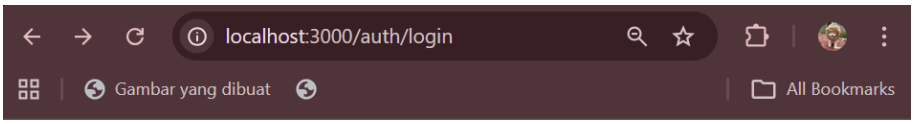
Manfaat:

- Routing tetap bersih
- Logic & UI terpisah
- Mudah dikembangkan

6. Inline Styling (CSS-in-JS)
   - Modifikasi index.tsx pada folder views/auth/login
     <h1 style={{color:"red",border:"1px solid red",borderRadius:"5px",padding:"5px"}}> belum punya akun</h1>

```jsx
import Link from "next/link";
import { useRouter } from "next/router";
import styles from "./login.module.css";

const TampilanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // Simpan status login
        localStorage.setItem("isLogin", "true");

        // logic login disini
        push("/produk");
    }
    return (
        <div className={styles.login}>

            <h1>Halaman Login</h1>

            {/* <button onClick={handleLogin}>Login</button> <br /> */}
            {/* <button onClick={() => push('/produk')}>Login</button> <br /> */}

            {/* Navigasi Imperatif */}
            <button onClick={() => handleLogin()}>Login</button> <br />
            <h1 style={{color:"red",border:"1px solid red",borderRadius:"5px",padding:"5px"}}> belum punya akun</h1>

            {/* Navigasi Link */}
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default TampilanLogin;
```

- Jalankan browser



Catatan Penting:

- Menggunakan camelCase
  borderRadius: "10px",
- Cocok untuk styling kecil & dinamis

- Tidak disarankan untuk layout besar

7. Kombinasi Global CSS + CSS Module
   - Modifikasi global.css

```
globals.css M ×   index.tsx ...\pages M   index.tsx ...\login U   login.
my-app-05 > src > styles > globals.css > ...
1
2
3   * {
4     box-sizing: border-box;
5     margin: 0;
6     padding: 0;
7   }
8
9   html,
10  body {
11    max-width: 100vw;
12    overflow: hidden;
13  }
14
15
16  a {
17    color: inherit;
18    text-decoration: none;
19  }
20
21  .big{
22    font-size: 1.5rem;
23  }
24
25
```

   - Modifikasi index.tsx pada folder components/layouts/navbar

```
gin.module.css U   index.tsx ...\Appshell M   index.tsx ...\components M ×
my-app-05 > src > components > index.tsx > Navbar
1   import syles from './navbar.module.css';
2
    Windsurf: Refactor | Explain | Generate JSDoc | ×
3   const Navbar = () => {
4       return (
5       <div className={syles.navbar}>
6           <div className="big">
7               Navbar
8       </div>
9   </div>
10      );
11  };
12
13  export default Navbar;
```

Kesimpulan:

- Global → utility umum

- Module → styling spesifik komponen

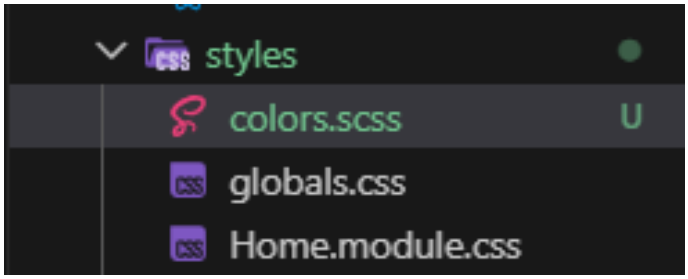
8. SCSS (SASS)
   a. Install SASS

```
PS D:\Pemograman_Framework\praktikum_05\my-app-05> npm i --save-dev sass
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: 'eslint-visitor-keys@5.0.1',
npm WARN EBADENGINE   required: { node: '^20.19.0 || ^22.13.0 || >=24' },
npm WARN EBADENGINE   current: { node: 'v20.11.1', npm: '10.2.4' }
npm WARN EBADENGINE }

added 8 packages, and audited 350 packages in 5s

142 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

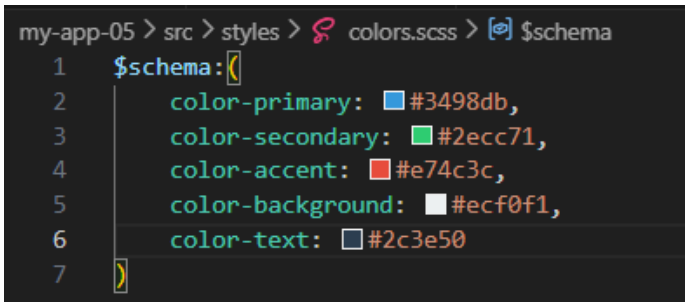
   - Cek pada package.css jika berhasil mak akan muncul seperti pada gambar

```
my-app-05 > package.json > { } devDependencies
1   {
2     "name": "my-app",
3     "version": "0.1.0",
4     "private": true,
      ▷Debug
5     "scripts": {
6       "dev": "next dev",
7       "build": "next build",
8       "start": "next start",
9       "lint": "next lint"
10    },
11    "dependencies": {
12      "@types/node": "25.2.3",
13      "@types/react": "19.2.14",
14      "@types/react-dom": "19.2.3",
15      "eslint": "9.39.2",
16      "eslint-config-next": "16.1.6",
17      "next": "16.1.6",
18      "react": "19.2.4",
19      "react-dom": "19.2.4",
20      "typescript": "5.9.3"
21    },
22    "devDependencies": {
23      "sass": "^1.97.3"
24    }
25  }
26
```

   b. Global Variable
   - Tambahkan colors.scss pada folder styles

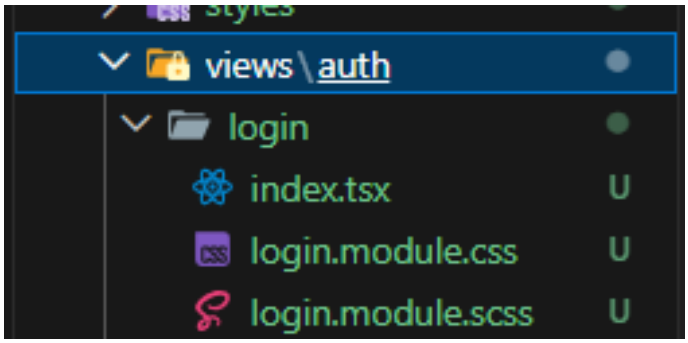


- Modifikasi colors.scss



c. Gunakan di Module

- Tambahkan file login.module.scss pada folder views/auth/login/



- Modifikasi index.tsx

  Tambahkan import styles from login.module.scss

  Disable import styles from login.module.css

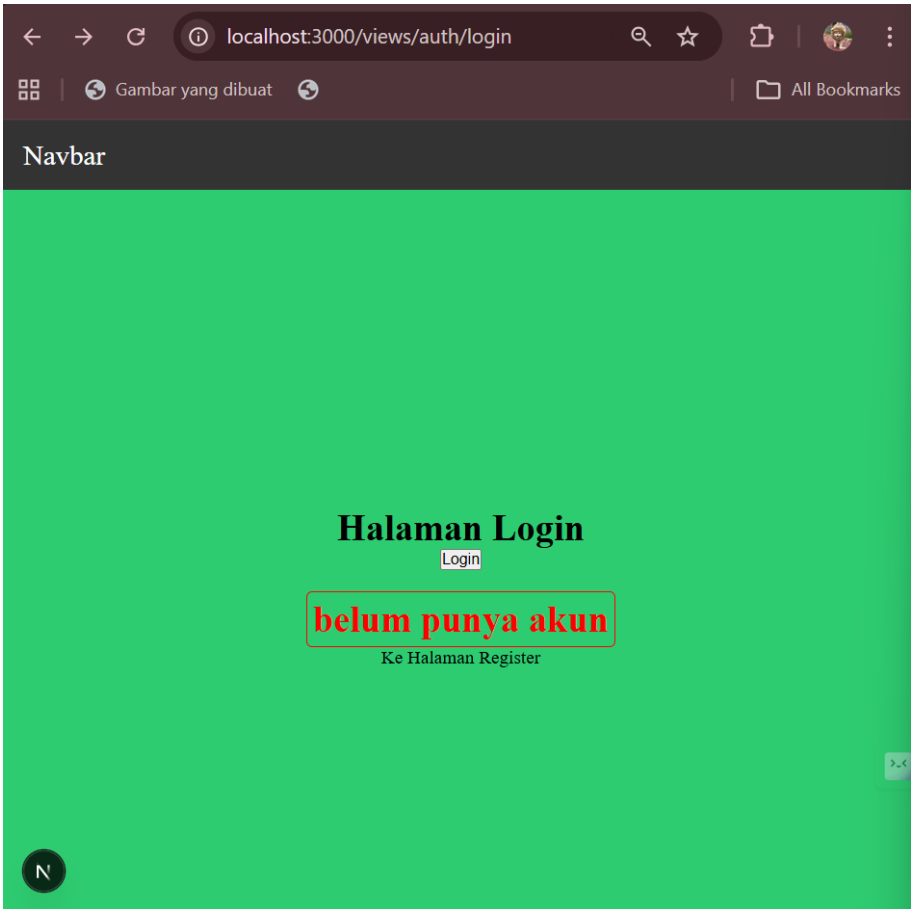
```
import Link from "next/link";
import { useRouter } from "next/router";
// import styles from "./login.module.css";
import styles from 'login.module.scss';

const TampilanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // Simpan status login
        localStorage.setItem("isLogin", "true");

        // logic login disini
        push("/produk");
    }
    return (
        <div className={styles.login}>

            <h1>Halaman Login</h1>

            {/* <button onClick={handleLogin}>Login</button> <br /> */}
            {/* <button onClick={() => push('/produk')}>Login</button> <br /> */}

            {/* Navigasi Imperatif */}
            <button onClick={() => handleLogin()}>Login</button> <br />
            <h1 style={{color:"red",border:"1px solid red",borderRadius:"5px",padding:"5px"}}> belum punya akun</h1>

            {/* Navigasi Link */}
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default TampilanLogin;
```

- Modifikasi login.module.scss

  background-color: map-get($map: $schema, $key:color-secondary);

```
my-app-05 > src > pages > views > auth > login > login.module.scss > .login
@import "@/styles/colors.scss";

.login{
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    height: 100vh;
    background-color: map-get($map: $schema, $key: color-secondary);
}
```

- Jalankan browser



9. Tailwind CSS
   a. Install
      - npm install -D tailwindcss postcss autoprefixer

```
PS D:\Pemograman_Framework\praktikum_05> npm i -D tailwindcss postcss auto
prefixer

added 15 packages in 5s

7 packages are looking for funding
  run `npm fund` for details
```

   b. Konfigurasi tailwind.config.js

```
my-app-05 > tailwind.config.js > <unknown> > theme
1   /** @type {import('tailwindcss').Config} */
2   module.exports = {
3       content: [
4           "./app/**/*.{js,ts,jsx,tsx,mdx}",
5           "./pages/**/*.{js,ts,jsx,tsx,mdx}",
6           "./components/**/*.{js,ts,jsx,tsx,mdx}",
7           "./src/**/*.{js,ts,jsx,tsx,mdx}",
8       ],
9       theme: {
10          extend: {},
11      },
12      plugins: [],
13  }
```

c. Import di Global CSS

```
1   @tailwind base;
2   @tailwind components;
3   @tailwind utilities;
4
5   * {
6     /* box-sizing: border-box;
7     margin: 0;
8     padding: 0; */
9   }
10
11  html,
12  body {
13    /* max-width: 100vw;
14    overflow: hidden; */
15  }
16
17
18  a {
19    /* color: inherit;
20    text-decoration: none; */
21  }
22
23  .big{
24    /* font-size: 1.5rem; */
25  }
26
27
```

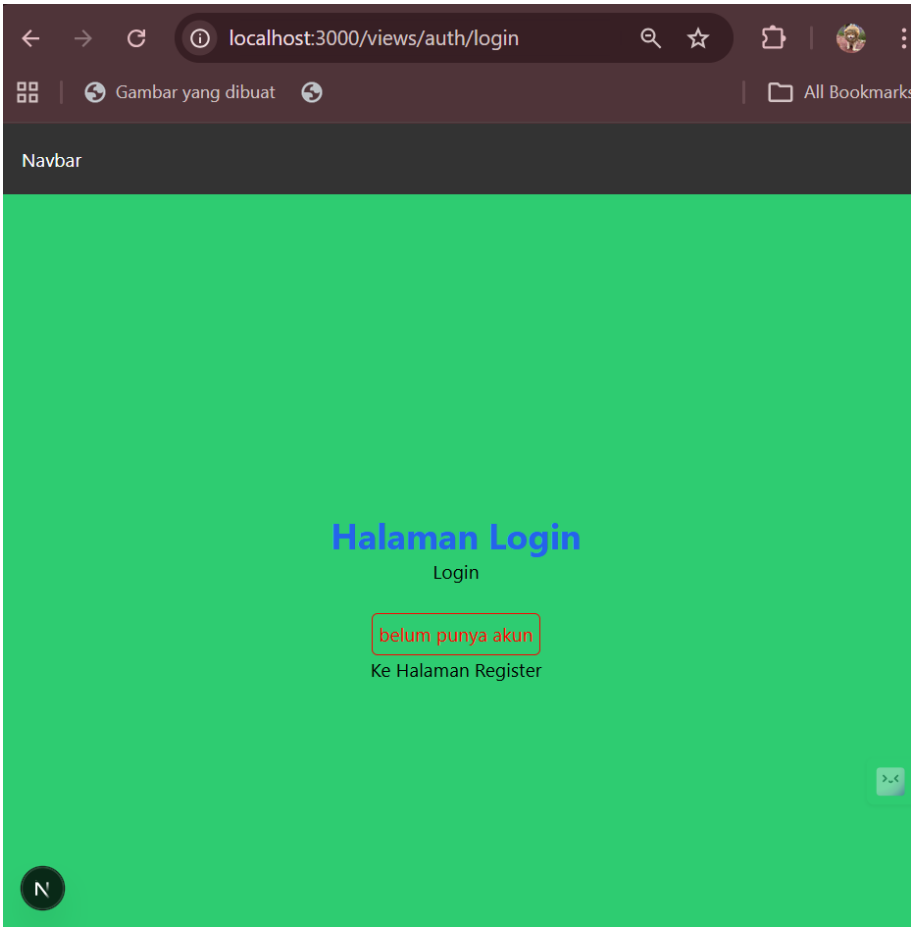
d. Contoh Penggunaan

- Modifikasi index.tsx pada folder auth/login/

<h1 className="text-3xl font-bold text-blue-600 ">Halaman Login</h1>

```
import Link from "next/link";
import { useRouter } from "next/router";
// import styles from "./login.module.css";
import styles from './login.module.scss';

const TampilanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // Simpan status login
        localStorage.setItem("isLogin", "true");

        // logic login disini
        push("/produk");
    }
    return (
        <div className={styles.login}>

            <h1 className="text-3xl font-bold text-blue-600 ">Halaman Login</h1>

            {/* <button onClick={handleLogin}>Login</button> <br /> */}
            {/* <button onClick={() => push('/produk')}>Login</button> <br /> */}

            {/* Navigasi Imperatif */}
            <button onClick={() => handleLogin()}>Login</button> <br />
            <h1 style={{color:"red",border:"1px solid red",borderRadius:"5px",padding:"5px"}}> belum punya akun</h1>

            {/* Navigasi Link */}
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default TampilanLogin;
```

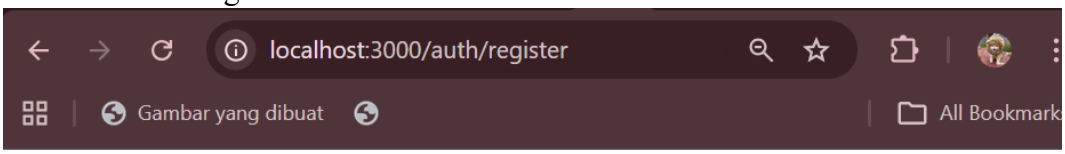
- Jalankan browser

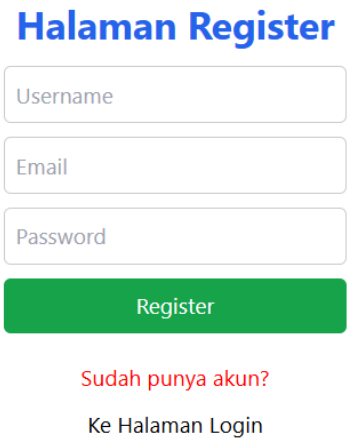


E. Tugas Praktikum

Tugas 1

a) Buat halaman Register





b) Gunakan CSS Module

c) fgaiiadbbb

- pages/auth/register
- ijffijifjf

register.tsx

```
import TampilanRegister from "@/views/auth/register";

const HalamanRegister = () => {
  return (
    <>
      <TampilanRegister />
    </>
  );
};

export default HalamanRegister;
```

- views/auth/register
  index.tsx

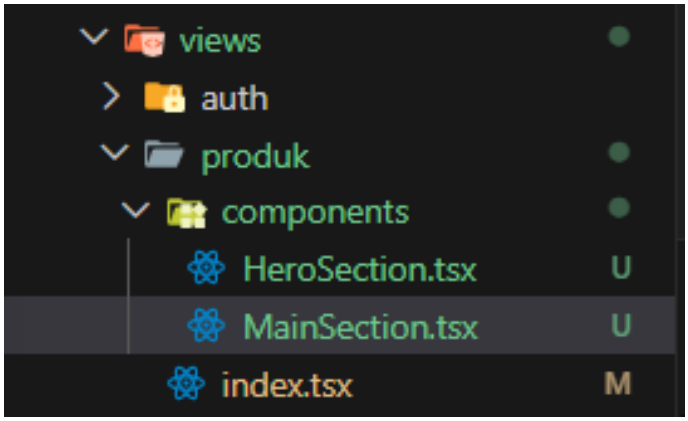
```python
import Link from "next/link";
import { useRouter } from "next/router";
import styles from "./register.module.css";

const TampilanRegister = () => {
  const { push } = useRouter();

  const handleRegister = () => {
    // Simulasi register berhasil
    alert("Register Berhasil!");

    // Setelah register arahkan ke login
    push("/auth/login");
  };

  return (
    <div className={styles.register}>
      <h1 className={styles.title}>Halaman Register</h1>

      <input
        type="text"
        placeholder="Username"
        className={styles.input}
      />

      <input
        type="email"
        placeholder="Email"
        className={styles.input}
      />

      <input
        type="password"
        placeholder="Password"
        className={styles.input}
      />

      <button onClick={handleRegister} className={styles.button}>
        Register
      </button>

      <p className={styles.text}>
        Sudah punya akun?
      </p>

      <Link href="/auth/login">Ke Halaman Login</Link>
    </div>
  );
};

export default TampilanRegister;
```

register.module.css

```css
.register {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  gap: 10px;
  height: 100vh;
}

.title {
  font-size: 28px;
  font-weight: bold;
  color: #2563eb;
}

.input {
  width: 250px;
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

.button {
  width: 250px;
  padding: 8px;
  background-color: #16a34a;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

.button:hover {
  background-color: #15803d;
}

.text {
  margin-top: 10px;
  color: red;
}
```

Tugas 2

a) Refactor halaman Produk ke folder views



b) Pisahkan Hero Section dan Main Section

- HeroSection.tsx

```
const HeroSection = () => {
  return (
    <section style={{
      backgroundColor: "#2563eb",
      color: "white",
      padding: "40px",
      textAlign: "center"
    }}>
      <h1>Produk</h1>
      <p>Selamat datang di halaman produk</p>
    </section>
  );
};

export default HeroSection;
```

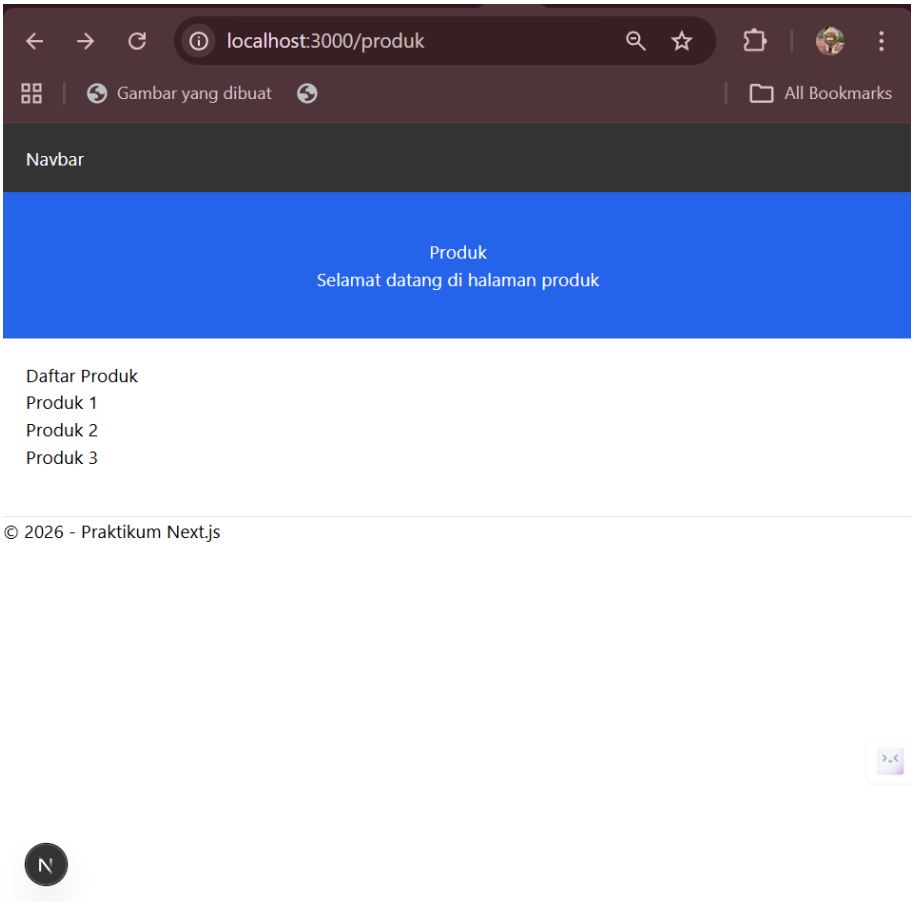
- MainSection.tsx

```
import Link from "next/link";

const MainSection = () => {
  return (
    <section style={{ padding: "20px" }}>
      <h2>Daftar Produk</h2>
      <ul>
        <li>
          <Link href="/produk/1">Produk 1</Link>
        </li>
        <li>
          <Link href="/produk/2">Produk 2</Link>
        </li>
        <li>
          <Link href="/produk/3">Produk 3</Link>
        </li>
      </ul>
    </section>
  );
};

export default MainSection;
```

- index.tsx

```
import HeroSection from "./components/HeroSection";
import MainSection from "./components/MainSection";

const TampilanProduk = () => {
  return (
    <>
      <HeroSection />
      <MainSection />
    </>
  );
};

export default TampilanProduk;
```

- output



Tugas 3

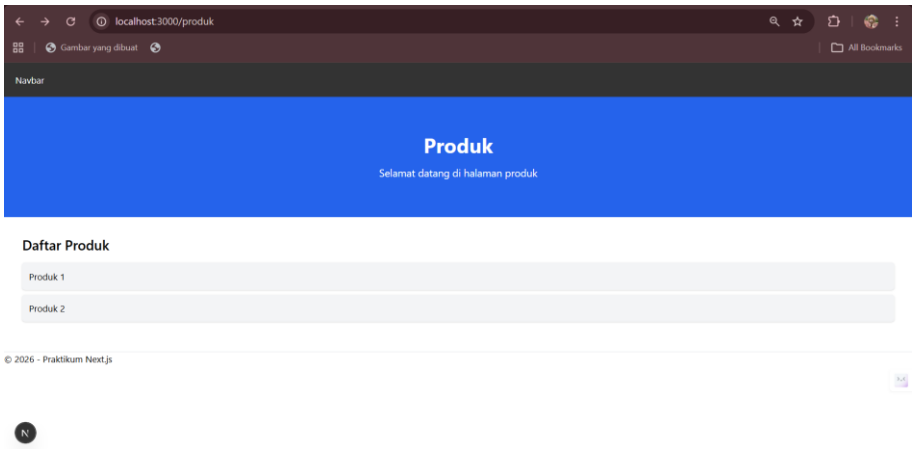
a) Terapkan Tailwind CSS
b) Gunakan minimal 5 utility class
   - HeroSection.tsx

```
const HeroSection = () => {
  return (
    <section className="bg-blue-600 text-white py-16 text-center">
      <h1 className="text-4xl font-bold mb-4">
        Produk
      </h1>
      <p className="text-lg">
        Selamat datang di halaman produk
      </p>
    </section>
  );
};

export default HeroSection;
```

- MainSection.tsx

```jsx
import Link from "next/link";

const MainSection = () => {
  return (
    <section className="p-8">
      <h2 className="text-2xl font-semibold mb-4">
        Daftar Produk
      </h2>

      <ul className="space-y-2">
        <li className="bg-gray-100 p-3 rounded-md shadow hover:bg-gray-200">
          <Link href="/produk/1" className="block">
            Produk 1
          </Link>
        </li>

        <li className="bg-gray-100 p-3 rounded-md shadow hover:bg-gray-200">
          <Link href="/produk/2" className="block">
            Produk 2
          </Link>
        </li>
      </ul>
    </section>
  );
};

export default MainSection;
```

- Output:



F. Pertanyaan Refleksi

1. Kapan sebaiknya menggunakan CSS Module dibanding Global CSS?
   - CSS Module sebaiknya digunakan ketika ingin membuat styling yang hanya berlaku untuk satu komponen tertentu saja. Dengan CSS Module, class yang dibuat tidak akan bentrok dengan class di file lain karena sifatnya lokal. CSS Module cocok digunakan pada project yang memiliki banyak komponen agar struktur styling lebih rapi dan terorganisir. Sedangkan Global CSS biasanya digunakan untuk styling umum seperti reset CSS, font, atau layout utama yang dipakai di seluruh halaman.
2. Apa kelemahan inline styling?
   - Inline styling memiliki beberapa kelemahan, yaitu:
     - Tidak bisa menggunakan fitur CSS lanjutan seperti hover, media query, atau pseudo-class.
     - Kode menjadi kurang rapi jika terlalu banyak style ditulis langsung di dalam tag.
     - Sulit digunakan kembali.
     - Kurang efisien jika project semakin besar.

     Karena itu, inline styling biasanya hanya digunakan untuk kondisi tertentu saja, bukan untuk styling utama.

3. Mengapa SCSS cocok untuk project skala besar?
   - SCSS cocok untuk project skala besar karena memiliki fitur yang lebih lengkap dibanding CSS biasa, seperti:
     - Variabel
     - Nested selector
     - Function dan mixin
     - Import file yang lebih terstruktur

     Dengan fitur tersebut, kode menjadi lebih terorganisir, mudah dibaca, dan mudah dikelola. Hal ini sangat membantu ketika project memiliki banyak file dan banyak komponen.

4. Apa keunggulan Tailwind dibanding CSS tradisional?
   - Keunggulan Tailwind dibanding CSS tradisional adalah karena menggunakan pendekatan utility-first. Styling langsung dituliskan pada className tanpa perlu membuat file CSS tambahan.

     Beberapa keunggulan Tailwind antara lain:
     - Proses styling lebih cepat
     - Tidak perlu membuat banyak class custom
     - Mengurangi konflik antar class
     - Lebih konsisten karena menggunakan sistem desain bawaan

     Dengan Tailwind, pengembangan tampilan menjadi lebih praktis dan efisien, terutama pada project berbasis komponen seperti React atau Next.js.